ShrinkageTrees: An R Package for Bayesian Tree Ensembles for Survival Analysis and Causal Inference

by Tijn Jacobs, Stéphanie van der Pas, and Wessel van Wieringen

Abstract ShrinkageTrees is an R package for Bayesian tree ensembles in survival analysis and causal inference. The package implements Bayesian additive regression tree models for right- and interval-censored survival outcomes within an accelerated failure time (AFT) framework, with optional decomposition into prognostic and treatment-effect components for causal inference. Two complementary forms of regularisation are available: regularisation of the tree structure, via depth-penalising priors and Dirichlet splitting priors, and regularisation of the leaf step heights, via global–local scale mixture priors. ShrinkageTrees provides the first implementation of the Horseshoe Forest, which places a horseshoe prior on the leaf step heights. These regularisation strategies make the package applicable to high-dimensional settings. An efficient Rcpp backend, multi-chain MCMC, and S3 methods for prediction, summary, and convergence diagnostics support the full modelling workflow, from fitting to posterior inference and diagnostics.

1 Introduction

Survival analysis in high-dimensional settings poses three interrelated challenges. First, the relationship between covariates and the time-to-event outcome is often complex and unknown, calling for flexible non-parametric models. Second, the outcome may be observed imprecisely: right-censored when a patient is lost to follow-up, or interval-censored when the event is known only to have occurred between two assessment times. Third, when a treatment is involved, its effect may vary across patient subgroups — and disentangling this heterogeneous treatment effect from prognostic factors requires dedicated methodology. These challenges arise routinely in clinical trials, observational studies, and genomics, often simultaneously.

Bayesian Additive Regression Trees [BART; Chipman et al. (2010)] offer a natural starting point. BART models the response as a sum of many shallow decision trees, where each tree contributes a small piece of the overall prediction. The ensemble captures non-linearities and interactions without requiring a parametric form. Existing BART implementations leave gaps in each of the three areas above. Currently, no R package provides Bayesian tree ensembles for interval-censored survival outcomes. The τ-learner — a dedicated causal forest architecture that separates the prognostic function from the treatment effect (Hahn et al., 2020; Caron et al., 2022) — has not been implemented for time-to-event endpoints. Standard BART's regularisation, while adequate in moderate dimensions, does not adapt well to the high-dimensional settings ($p \gg n$) common in genomics.

ShrinkageTrees addresses all three gaps. The package fits Bayesian tree ensembles to right- and interval-censored survival data via accelerated failure time (AFT) models, with optional τ-learner decomposition for causal inference. The package implements two complementary regularisation strategies for high-dimensional settings. Regularisation of the tree structure encourages the ensemble to split on a subset of relevant variables. The package implements the Dirichlet splitting approach of DART (Linero, 2018). Regularisation of the leaf step heights targets the contributions of individual leaves: it shrinks uninformative leaves towards zero while preserving strong signals. ShrinkageTrees provides the first implementation of the Horseshoe Forest (Jacobs et al., 2025), which places a horseshoe prior (Carvalho et al., 2010) on the leaf step heights to adaptively shrink uninformative leaves.

The package is implemented with an efficient C++ backend via **Rcpp** (Eddelbuettel and François, 2011) and provides a coherent S3 interface with `print`, `summary`, `predict`, and `plot` methods. The paper is aimed at both practitioners encountering Bayesian tree ensembles for the first time and experienced users seeking functionality not available in existing packages.

To give a first impression, we use the `ovarian` dataset shipped with the package: a semi-synthetic cohort with covariates drawn from The Cancer Genome Atlas ovarian cancer cohort [TCGA-OV; Cancer Genome Atlas Research Network (2011)] and survival outcomes simulated from a known data-generating mechanism. It comprises 357 patients with right-censored overall survival, a binary treatment indicator, four clinical covariates, and expression levels of 1,000 genes — a high-dimensional ($p \gg n$) setting that exercises the main features of the package. A Horseshoe Forest can then be fitted in a single call:

```
library(ShrinkageTrees)
```

```
data("ovarian")
time   <- ovarian$OS_time / 30.44  # days to months
status <- ovarian$OS_event
X      <- as.matrix(ovarian[, -(1:3)])
fit    <- HorseTrees(y = time, status = status, X_train = X,
                     outcome_type = "right-censored", number_of_trees = 200)
```

The remainder of this paper proceeds as follows. Section 2 presents a worked example on an ovarian-cancer dataset, exposing the overfitting that standard BART suffers in $p \gg n$ and motivating the horseshoe. Section 3 reviews the statistical methodology. Section 4 returns to the example, adds horseshoe shrinkage, demonstrates the τ-learner for treatment effects, and validates the package on simulated interval-censored data. Section 5 describes the package architecture. Section 6 compares **ShrinkageTrees** with related R packages. Section 7 concludes.

2 A worked example: Survival prediction on the ovarian dataset

We introduce the package through a complete worked example before discussing the underlying methodology. We return to the `ovarian` dataset loaded in the introduction. The treatment indicator distinguishes carboplatin from cisplatin, and the four clinical covariates are age, FIGO stage, tumour grade, and year of diagnosis. The 1,000 gene expression features are selected by median absolute deviation. Ovarian cancer is one of the most lethal gynaecological malignancies, with a five-year survival rate below 50%, and gene expression patterns are known to be associated with survival duration and chemotherapy response (Cancer Genome Atlas Research Network, 2011). We additionally extract the treatment indicator, used in Section 4.2.

```
treatment <- ovarian$treatment
```

We begin with the simplest model available in **ShrinkageTrees**: a standard BART survival model via `SurvivalBART()`. This uses an accelerated failure time (AFT) formulation with right-censoring and a conjugate normal prior on the leaf step heights — the classical BART specification of Chipman et al. (2010) applied to survival data. We fit the model on the full cohort.

```
fit_bart <- SurvivalBART(
  time           = time,
  status         = status,
  X_train        = X,
  timescale      = "time",
  number_of_trees = 200,
  N_post         = 5000,
  N_burn         = 5000,
  n_chains       = 4
)
```

The `print()` method provides a concise overview of the fitted model.

```
print(fit_bart)
```

```
ShrinkageTrees model
---------------------
Outcome type:         Right-censored (AFT, timescale = time)
Prior:                standard
Number of trees:      200
Training size (n):    357
Number of features:   1004
Chains:               4
Draws per chain:      5000 (burn-in 5000)
Acceptance ratio:     0.244, 0.243, 0.247, 0.239 (per chain)
Posterior mean sigma: 0.23
```

We evaluate in-sample fit using the concordance index [C-index; Harrell et al. (1996)], where a value of 1 indicates perfect discrimination and 0.5 corresponds to random prediction.

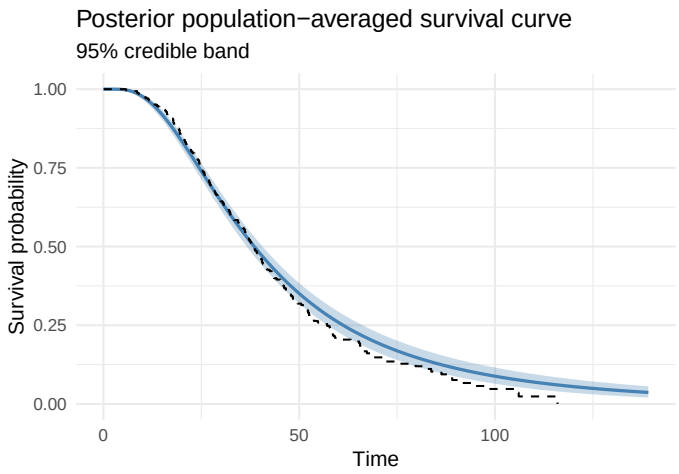

Figure 1: Population-averaged posterior survival curve under the standard BART baseline (solid line) with 95% credible band (shaded) and Kaplan–Meier estimate (dashed).

```
library(survival)
c_train <- concordance(Surv(time, status) ~ fit_bart$train_predictions)
cat("Train C-index:", round(c_train$concordance, 3), "\n")
```

```
Train C-index (SurvivalBART): 0.989
```

The training C-index is close to one: standard BART discriminates the observed events almost perfectly. With only $n = 357$ patients and $p = 1{,}004$ predictors, this near-perfect in-sample fit is a clear sign of overfitting — the conjugate normal prior on the leaf step heights does not provide enough regularisation in this $p \gg n$ regime. Stronger regularisation is needed. **ShrinkageTrees** offers several options to that end, including a Dirichlet splitting prior on the tree structure (DART) and global–local shrinkage priors on the leaf step heights, of which the horseshoe is the headline contribution. We introduce these options in Section 3 and revisit the ovarian example with horseshoe shrinkage in Section 4.1.

The `plot()` method produces posterior survival curves. The population-averaged curve across all observations can be overlaid with the Kaplan–Meier estimate for visual calibration:

```
plot(fit_bart, type = "survival", km = TRUE)
```

Figure 1 shows the population-averaged curve tracking the Kaplan–Meier estimate closely — unsurprising given the near-perfect in-sample C-index, but the apparent in-sample fit comes at the cost of overfitting, as Section 4.1 will make clear. Before returning to this dataset in Section 4 — where we progressively add horseshoe shrinkage, causal inference, and interval-censored outcomes — we review the statistical methodology that underpins these extensions.

3 Statistical methodology

Having seen **ShrinkageTrees** in action, we now formalise the models behind it. We build on three components. First, we review the BART model and the AFT formulation for survival outcomes — the model fitted by `SurvivalBART()` in Section 2. Second, we show how regularisation targets either the tree structures or the step heights. Third, we extend the framework to causal inference via the τ-learner.

We use the following notation throughout. The data consist of n subjects indexed by $i = 1, \ldots, n$, each with a covariate vector $\mathbf{x}_i \in \mathbb{R}^p$ and a response Y_i (continuous in the BART subsection, $\log T_i$ in the AFT subsection). An ensemble consists of m trees indexed by $j = 1, \ldots, m$. Bold lowercase letters denote vectors and calligraphic letters denote tree-related objects: $\mathcal{T}_j$ for the tree structure and $\mathcal{H}_j$ for its step heights.

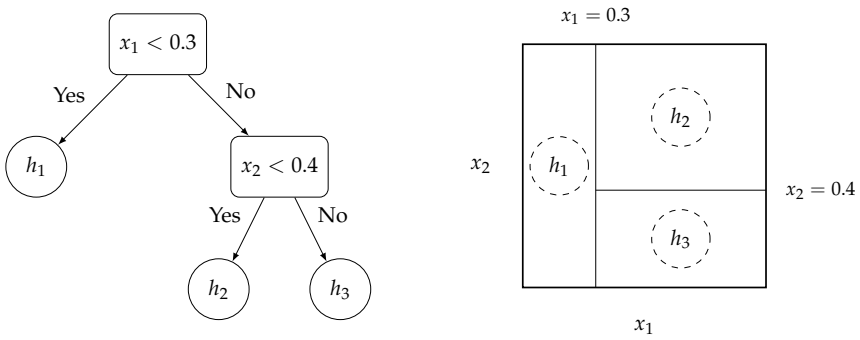

Figure 2: Schematic of a single regression tree. Interior nodes contain binary splitting rules of the form $x_\rho < c$; terminal nodes (leaves) contain step heights h_ℓ. An observation traverses the tree from root to leaf, and its prediction is the step height of the leaf it reaches.

BART and the accelerated failure time model

BART (Chipman et al., 2010) models the response as a sum of m shallow decision trees:

$$Y_i = \sum_{j=1}^{m} g(\mathbf{x}_i;\, \mathcal{T}_j, \mathcal{H}_j) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

Each tree j is characterised by two groups of parameters. The *tree structure* $\mathcal{T}_j$ encodes the topology and the splitting rules: which variables to split on and at what thresholds. It determines how the covariate space is partitioned into disjoint regions. The *step heights* $\mathcal{H}_j = \{h_{j1}, \ldots, h_{jL_j}\}$ are the values assigned to the L_j leaves and determine each region's contribution to the fitted function. For an input $\mathbf{x}$, the function $g(\mathbf{x};\, \mathcal{T}_j, \mathcal{H}_j)$ returns the step height of the leaf that $\mathbf{x}$ reaches (see Figure 2). This distinction between $\mathcal{T}$ and $\mathcal{H}$ is central to the regularisation framework we develop below.

We specify priors that are independent across trees and factorise within each tree:

$$p(\{\mathcal{T}_j, \mathcal{H}_j\}_{j=1}^{m}) = \prod_{j=1}^{m} p_{\mathcal{T}}(\mathcal{T}_j)\, p_{\mathcal{H}}(\mathcal{H}_j \mid \mathcal{T}_j).$$

The tree structure prior $p_{\mathcal{T}}$ is a branching process (Chipman et al., 1998; Ročková and Saha, 2019) in which a node at depth d splits with probability $\alpha\,(1+d)^{-\beta}$, with $\alpha \in (0,1)$ and $\beta \geq 0$. The default values $\alpha = 0.95$ and $\beta = 2$ keep most trees shallow. At each internal node, the splitting variable is drawn uniformly from the p covariates and the split point uniformly from the observed range of that variable.

The original BART formulation $p_{\mathcal{H}}$ assigns Gaussian priors to the step heights, independent across leaves and trees, with variance $\sigma_h^2 = 1/(4k^2 m)$ on the centred and scaled response. The default $k = 2$ places approximately 95% of the prior mass within the observed range, following Chipman et al. (2010).

We model the error variance as $\sigma^2 \sim \text{Inv-}\chi^2(\nu,\, \psi)$, with $\nu = 3$ and ψ chosen so that the prior is centred around a rough estimate of σ^2 (Chipman et al., 2010).

For survival outcomes, we adopt an accelerated failure time (AFT) formulation. Let T_i denote the event time, C_i the censoring time, $Y_i = \min(T_i, C_i)$ the observed follow-up time, and $\delta_i \in \{0,1\}$ the event indicator. The prediction model becomes:

$$\log T_i = \sum_{j=1}^{m} g(\mathbf{x}_i;\, \mathcal{T}_j, \mathcal{H}_j) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

For right-censored observations ($\delta_i = 0$), we augment $\log T_i$ from a truncated normal distribution within the Gibbs sampler. **ShrinkageTrees** also supports interval-censored outcomes, where the event time is known only to lie in an interval $(l_i, r_i]$. We augment these analogously. This is the model behind the `SurvivalBART()` call in Section 2.

Regularisation

The parameters of a BART ensemble fall into two groups: tree structure parameters (nodes, splitting variables, split points) that determine how the covariate space is partitioned, and step heights that determine each partition's contribution to the fitted function. Regularisation can target either group.

Tree structure regularisation. The branching process prior on $\mathcal{T}_j$ is itself a regularisation mechanism: through α and β, it discourages deep trees. DART (Linero, 2018) adds a second mechanism. Standard BART draws splitting variables uniformly from the p covariates. DART instead places a Dirichlet prior on the splitting probabilities $\mathbf{s} = (s_1, \ldots, s_p)$:

$$\mathbf{s} \sim \text{Dirichlet}\left(\frac{\theta}{p}, \ldots, \frac{\theta}{p}\right).$$

Smaller values of θ concentrate the prior on sparse probability vectors, so the ensemble splits on a subset of relevant variables. This is a form of structural variable selection. We follow the hyperprior calibration of Linero (2018) and place a hyperprior on θ:

$$\frac{\theta}{\theta + \rho} \sim \text{Beta}(a, b),$$

with defaults $a = 0.5$, $b = 1$, and $\rho = p$.

Shrinkage on step heights. Regularisation can also target the leaf step heights directly. **ShrinkageTrees** adopts a scale mixture of normals representation:

$$h_\ell \mid \lambda_\ell, \tau, \omega \sim \mathcal{N}\left(0,\, \omega\, \lambda_\ell^2\, \tau^2\right),$$

where τ is a global shrinkage parameter shared across all leaves in a tree, λ_ℓ is a local scale specific to leaf ℓ, and ω is a fixed scaling constant. The global parameter τ controls the overall degree of shrinkage, while the local parameters λ_ℓ allow individual leaves with strong signals to escape shrinkage.

The horseshoe prior (Carvalho et al., 2010) is a specific instance of this framework, with independent half-Cauchy priors on the local and global scales:

$$\lambda_\ell \sim \text{C}^+(0, \alpha_\lambda), \qquad \tau \sim \text{C}^+(0, \alpha_\tau),$$

where $\text{C}^+(0, \alpha)$ denotes the half-Cauchy distribution with scale α. The half-Cauchy has its median at its scale parameter and heavy tails. Its mean and variance do not exist, which allows the local scales to take very large values when the data support a strong effect (Polson and Scott, 2012). The combination shrinks the contributions of uninformative leaves towards zero while preserving large, data-supported signals.

We set $\alpha_\tau = \alpha_\lambda = k/\sqrt{m}$, following Jacobs et al. (2025). Smaller values of k yield stronger shrinkage. We recommend $k \in [0.05,\, 0.5]$ as a practical range. The default in **ShrinkageTrees** is $k = 0.1$, but the user can select k by cross-validation when domain knowledge does not suggest a specific value. Jacobs et al. (2025) provide a sensitivity analysis to support these recommendations.

Figure 3 maps the resulting design space. Four cells correspond to fitting functions exposed by the package, with `HorseTrees()` as the headline contribution. The two dashed cells combine Dirichlet splitting with local or global–local leaf shrinkage. These combinations are within the framework but are not yet exposed in the current release.

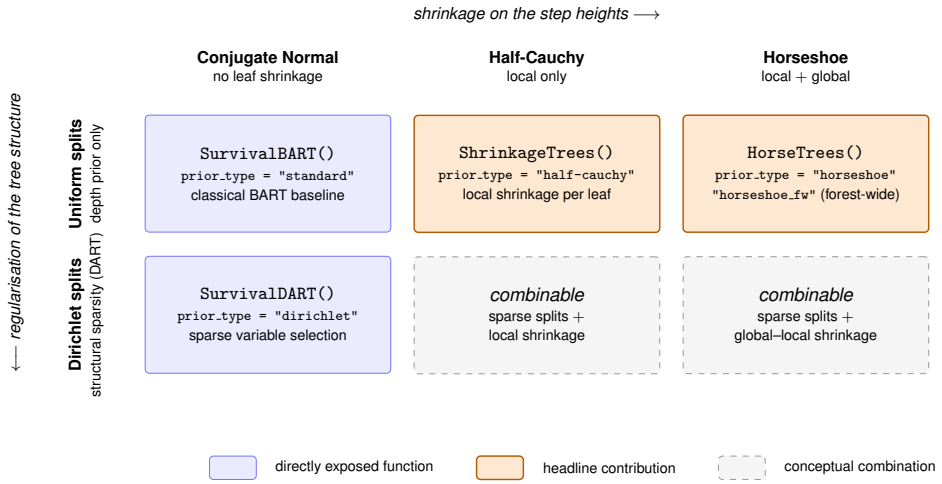

Figure 3: Regularisation landscape for **ShrinkageTrees**. Solid cells name the fitting function and `prior_type` value exposed in the package. Dashed cells mark combinations within the framework that the current release does not expose.

The τ-learner for causal inference

BART has become a popular tool for causal inference (Hill, 2011; Dorie et al., 2019). The BART formulation introduced earlier estimates a single function of the covariates, which we call the *prediction model*. Prediction models support two simple causal-inference workflows: an S-learner includes the treatment indicator as a covariate, and a T-learner fits separate models per treatment arm (Caron et al., 2022). **ShrinkageTrees** takes a different approach: the τ-learner. This is the decomposition proposed by Hahn et al. (2020) as Bayesian Causal Forests (BCF) and later termed the τ-learner by Caron et al. (2022). The **bcf** and **stochtree** packages implement the τ-learner for continuous outcomes. **ShrinkageTrees** extends it to survival endpoints. We write $A_i \in \{0,1\}$ for the binary treatment indicator and $\hat{e}(\mathbf{x}_i) = \hat{P}(A_i = 1 \mid \mathbf{x}_i)$ for the estimated propensity score. The model is:

$$y_i = \mu(\mathbf{x}_i, \hat{e}(\mathbf{x}_i)) + A_i\,\tau(\mathbf{x}_i) + \varepsilon_i,$$

where μ is a prognostic function modelled by a forest of m_μ trees and τ is a treatment effect function modelled by a separate forest of m_τ trees. This separation allows the prognostic and treatment effect components to have different regularisation and complexity. The conditional average treatment effect is $\mathrm{CATE}(\mathbf{x}) = \tau(\mathbf{x})$, and the average treatment effect is $\mathrm{ATE} = \mathbb{E}[\tau(\mathbf{X})]$.

The way treatment is coded in the τ-learner is a modelling convention with several common choices. Let b_i denote the coding applied to subject i. The model becomes:

$$y_i = \mu(\mathbf{x}_i, \hat{e}(\mathbf{x}_i)) + b_i\,\tau(\mathbf{x}_i) + \varepsilon_i.$$

Hahn et al. (2020) discuss several choices, all of which are implemented in **ShrinkageTrees** through the `treatment_coding` argument. **Centred** coding sets $b_i = A_i - 1/2$ and is the default. **Binary** coding sets $b_i = A_i$. **Adaptive** coding sets $b_i = A_i - \hat{e}(\mathbf{x}_i)$, the parameterisation used in the **bcf** package. The **invariant** (parameter-expanded) coding of Hahn et al. (2020, Section 5.2) is invariant to the labelling of treatment and control. It reparameterises the model with group-specific intercepts and recovers τ as a difference. We refer the reader to Hahn et al. (2020) for the full construction.

Posterior computation

All models are fitted using Markov chain Monte Carlo (MCMC). The outer loop is a Gibbs sampler that cycles through updates of the tree topologies, the leaf step heights, the error variance σ^2, and — where applicable — the shrinkage parameters and censored event times. Trees are updated one at a time via Bayesian backfitting (Hastie and Tibshirani, 2000): at each iteration, a partial residual is formed by subtracting the contributions of all other trees from the response, and tree j is then updated conditional on the current state of all other trees. The tree topology is modified through Metropolis–Hastings proposals that either grow a terminal node into an internal node with two new leaves, or prune an internal node back to a leaf. Details of the sampler are given in Kapelner and Bleich (2016).

Under the standard BART and Dirichlet (DART) priors, where the leaf parameters have a conjugate

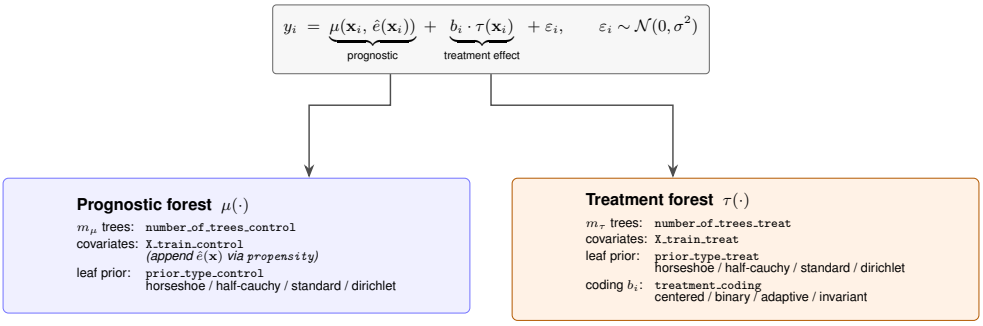

Figure 4: The τ-learner decomposition in **ShrinkageTrees**. The outcome splits into a prognostic forest $\mu(\mathbf{x}, \hat{e}(\mathbf{x}))$ (blue) and a treatment-effect forest $\tau(\mathbf{x})$ (orange). Each box lists the function arguments that control its forest. For survival outcomes, set $y_i = \log T_i$.

normal prior, the marginal likelihood of a proposed tree can be computed in closed form and the step heights are integrated out analytically. When global–local shrinkage priors such as the horseshoe are placed on the step heights, this conjugacy is lost: the local and global scale parameters λ_ℓ and τ must be sampled explicitly, and the tree proposals become reversible jump MCMC moves. The details of this reversible jump construction are given in Jacobs et al. (2025).

4 A closer look at the ovarian example

We return to the ovarian cancer dataset from Section 2 and progressively demonstrate more advanced features of **ShrinkageTrees**: horseshoe shrinkage for improved regularisation, causal inference for treatment effect estimation, and interval-censored survival modelling.

Horseshoe shrinkage

We now fit a Horseshoe Forest via `HorseTrees()` to the same data. The horseshoe prior applies global–local shrinkage to the leaf step heights, providing stronger regularisation than the conjugate normal prior used by `SurvivalBART()` — particularly beneficial in this high-dimensional setting where the number of predictors far exceeds the sample size.

```
fit_horse <- HorseTrees(
  y               = time,
  status          = status,
  X_train         = X,
  outcome_type    = "right-censored",
  timescale       = "time",
  number_of_trees = 200,
  k               = 0.1,
  N_post          = 5000,
  N_burn          = 5000,
  n_chains        = 4
)
```

The `print()` and `summary()` methods provide a concise overview of the fitted model, including the outcome type, MCMC settings, acceptance ratio, and posterior summaries for σ.

```
print(fit_horse)
```

```
ShrinkageTrees model
---------------------
Outcome type:         Right-censored (AFT, timescale = time)
Prior:                horseshoe
Number of trees:      200
Training size (n):    357
```

```
Number of features:   1004
Chains:               4
Draws per chain:      5000 (burn-in 5000)
Acceptance ratio:     0.505, 0.506, 0.505, 0.505 (per chain)
Posterior mean sigma: 0.993
```

```
summary(fit_horse)
```

```
ShrinkageTrees model summary
============================
Call: HorseTrees(y = time, status = status, X_train = X, outcome_type = "right-censored",
    timescale = "time", number_of_trees = 200, k = 0.1, N_post = 5000,
    N_burn = 5000, store_posterior_sample = TRUE, n_chains = 4,
    verbose = TRUE)

Outcome: Right-censored (AFT, timescale = time) | Prior: horseshoe | Trees: 200
Data:    n = 357, p = 1004 | Draws: 5000 x 4 chains (burn-in 5000)

Posterior sigma:
  Mean: 0.993  SD: 0.049  95% CI: [0.901, 1.093]

Predictions (posterior mean):
  Train: mean = 38.657, sd = 2.553, range = [31.837, 44.211]
  Test:  mean = 38.901, sd = NA, range = [38.901, 38.901]

MCMC acceptance ratio (per chain): 0.505, 0.506, 0.505, 0.505

Convergence diagnostics (coda):
  Gelman-Rubin R-hat:  = 1
  Effective sample size: sigma = 8918
```

We report the in-sample C-index for the horseshoe fit alongside the BART value from Section 2 for comparison. The first line of the output is repeated from Section 2. The second is new for the horseshoe fit.

```
c_train <- concordance(Surv(time, status) ~ fit_horse$train_predictions)
cat("Train C-index:", round(c_train$concordance, 3), "\n")
```

```
Train C-index (SurvivalBART): 0.989
Train C-index (HorseTrees):   0.727
```

The horseshoe produces a lower training C-index than `SurvivalBART()`. Standard BART concentrates ensemble capacity on whichever of the 1{,}004 covariates align with training events. The horseshoe shrinks contributions from leaves that are not strongly supported by the data. The simulation in Section 4.3 evaluates the two approaches on a controlled sparse setting where ground truth is available.

We assess convergence using σ traceplots and posterior densities. With `n_chains = 4`, each chain is plotted separately, allowing visual assessment of mixing and chain agreement. The `summary()` method automatically reports the effective sample size and Gelman–Rubin $\hat{R}$ statistic when the **coda** package is installed.

```
plot(fit_horse, type = "trace")
plot(fit_horse, type = "density")
```

Figure 5 shows good mixing: the four chains overlap throughout the sampling period and yield nearly identical posterior densities for σ.

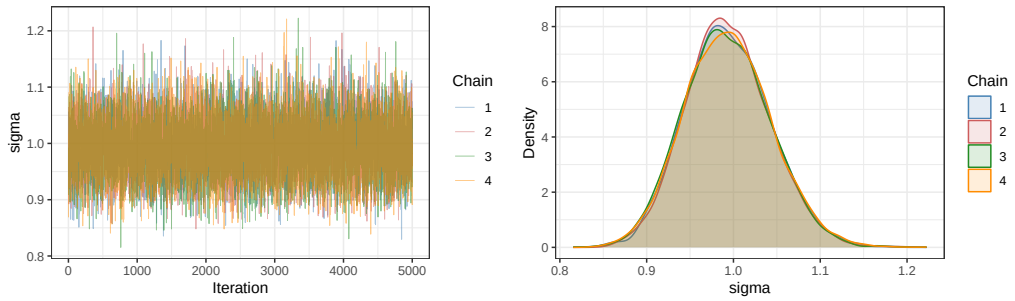

Figure 5: Left: traceplot of the posterior draws of σ for the Horseshoe Forest model. Right: posterior density of σ, estimated separately for each of the four chains.

We illustrate the posterior predictive survival curve for a randomly selected patient.

```
pred <- predict(fit_horse, newdata = X)
idx <- sample(length(pred$mean), 1)
```

The individual posterior survival curve under the AFT log-normal model $S(t \mid \mathbf{x}) = 1 - \Phi((\log t - \hat{\mu}(\mathbf{x}))/\hat{\sigma})$ is plotted with pointwise 95% credible bands.

```
plot(pred, type = "survival", obs = idx)
```

We also plot the population-averaged posterior survival curve across all training observations, overlaid with the Kaplan–Meier estimate for visual calibration.

```
plot(fit_horse, type = "survival", km = TRUE)
```

Figure 6 shows the individual and population-level posterior survival curves. The individual curve provides full uncertainty quantification for a single patient, while the population-averaged curve closely tracks the Kaplan–Meier estimate during the observed follow-up period, with expected divergence in the right tail where data are sparse.

Causal inference: treatment effects for survival

We now estimate heterogeneous treatment effects of carboplatin versus cisplatin using `CausalHorseForest()`, which fits a τ-learner with horseshoe shrinkage on both the prognostic and treatment effect forests. We first estimate propensity scores using a probit Horseshoe Forest on the clinical covariates. We choose a Horseshoe Forest over logistic regression because it handles potential non-linearities and interactions between clinical covariates without manual specification, while the horseshoe prior guards against overfitting in this low-sample-size setting. Using the full gene expression matrix for propensity estimation would risk overfitting given $p \gg n$. Clinical covariates alone capture the main sources of confounding by indication.

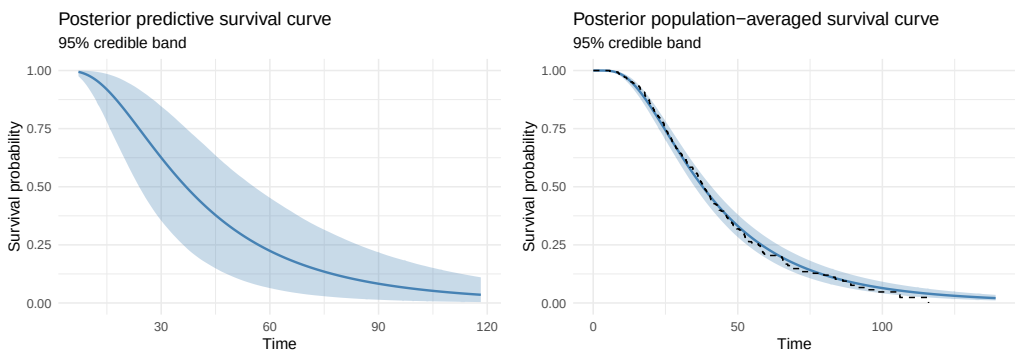

Figure 6: Left: posterior survival curve for a randomly selected patient, with pointwise 95% credible bands. Right: population-averaged posterior survival curve (solid line) with 95% credible band (shaded) and Kaplan–Meier estimate (dashed).

```
X_ps <- as.matrix(ovarian[, c("age", "figo_stage", "tumor_grade")])
ps_fit <- HorseTrees(
  y              = treatment,
  X_train        = X_ps,
  outcome_type   = "binary",
  number_of_trees = 200,
  k              = 0.1,
  N_post         = 5000,
  N_burn         = 5000
)
propensity <- ps_fit$train_predictions
X_control  <- cbind(propensity = propensity, X)

fit_causal <- CausalHorseForest(
  y                        = log(time),
  status                   = status,
  X_train_control          = X_control,
  X_train_treat            = X,
  treatment_indicator_train = treatment,
  outcome_type             = "right-censored",
  timescale                = "log",
  number_of_trees          = 200,
  k                        = 0.1,
  N_post                   = 5000,
  N_burn                   = 5000,
  n_chains                 = 4
)
summary(fit_causal)
```

```
CausalShrinkageForest model summary
====================================
Call: CausalHorseForest(y = log(time), status = status, X_train_control = X_control,
    X_train_treat = X, treatment_indicator_train = treatment,
    outcome_type = "right-censored", timescale = "log", number_of_trees = 200,
    k = 0.1, N_post = 5000, N_burn = 5000, store_posterior_sample = TRUE,
    n_chains = 4, verbose = TRUE)

Outcome: Right-censored (AFT, timescale = log)
Prior:   control = horseshoe, treatment = horseshoe
Trees:   control = 200, treatment = 200
Data:   n = 357, p_control = 1005, p_treat = 1004 | Draws: 5000 x 4 chains (burn-in 5000)

Treatment effect:
  PATE:    0.0211  95% CI (Bayesian bootstrap): [-0.0993, 0.1459]
  CATE SD: 0.0077

Prognostic function (mu):
  Mean: 3.63  SD: 0.011  Range: [3.604, 3.65]

Posterior sigma:
  Mean: 1.001  SD: 0.048  95% CI: [0.913, 1.101]

MCMC acceptance ratios (per chain):
  control:   0.508, 0.509, 0.509, 0.509
  treatment: 0.476, 0.475, 0.475, 0.475
```

The `summary()` method reports the posterior average treatment effect (ATE) on the log-survival scale with a 95% credible interval. Here we pass `log(time)` directly with `timescale = "log"`, so all treatment effect estimates remain on the log-survival scale. A positive ATE indicates that carboplatin is associated with longer survival times. By default, the reported interval is the population ATE obtained by a Bayesian bootstrap (Rubin, 1981) — at each MCMC iteration, the per-observation CATEs are reweighted with Dirichlet$(1, \ldots, 1)$ weights before averaging, so the credible interval propagates uncertainty in the covariate distribution F_X rather than conditioning on the sample. Setting

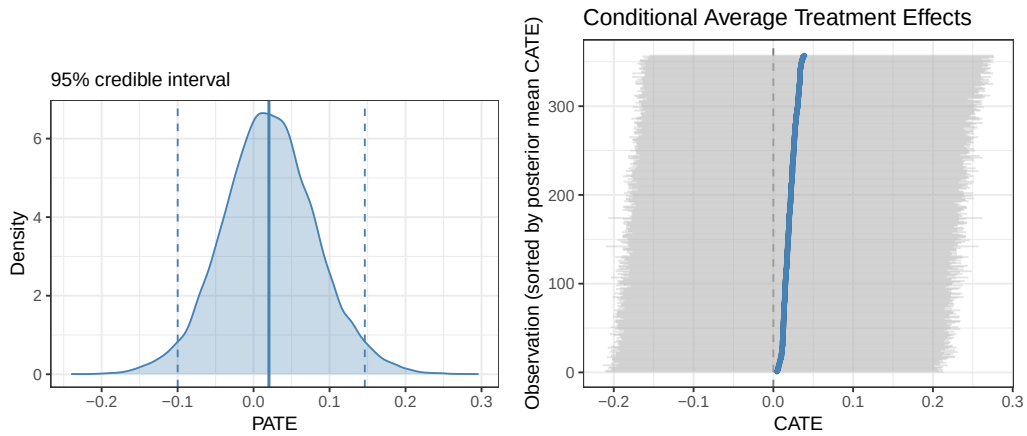

Figure 7: Left: posterior density of the average treatment effect (ATE) of carboplatin versus cisplatin on the log-survival scale, with 95% credible interval (dashed lines). Right: patient-level conditional average treatment effects (CATEs) sorted by posterior mean with 95% credible intervals. The dashed line marks zero (no effect).

`bayesian_bootstrap = FALSE` recovers the narrower mixed ATE [MATE; Li et al. (2023)], which conditions on the observed covariates and therefore tends to be overconfident.

The `plot()` method provides visualisations of the treatment effect posterior. The ATE density plot shows the full posterior distribution of the average treatment effect, while the CATE caterpillar plot displays patient-level treatment effect estimates sorted by posterior mean with 95% credible intervals.

```
plot(fit_causal, type = "ate")
plot(fit_causal, type = "cate")
```

Figure 7 shows the treatment effect posteriors. The posterior mean ATE is 0.065 on the log-survival scale, corresponding to an approximate 7% increase in log survival time under carboplatin relative to cisplatin. However, the 95% credible interval $[-0.10, 0.25]$ crosses zero, so the data do not provide clear evidence of a treatment effect. The individual CATEs cluster tightly around the ATE (CATE SD ≈ 0.01), suggesting a relatively homogeneous treatment effect across the patient population — consistent with both platinum agents acting through similar mechanisms.

Although the ovarian dataset contains right-censored observations, **ShrinkageTrees** also supports interval-censored outcomes via `outcome_type = "interval-censored"` with `left_time` and `right_time` arguments following the convention of `survival::Surv(type = "interval2")`. See `?ShrinkageTrees` for details and examples. We demonstrate this capability in the following simulation.

Interval-censored outcomes: a simulation study

We illustrate the interval-censored implementation of **ShrinkageTrees** through a controlled simulation with known ground truth. The data-generating process combines the non-linear regression benchmark of Friedman (1991) with a sparse high-dimensional linear term. Covariates are $X_{ij} \overset{\text{i.i.d.}}{\sim} \mathcal{U}[0,1]$ and the noiseless mean function is:

$$f(x_i) = 10\,\sin(\pi\, x_{i1}\, x_{i2}) + 20\,(x_{i3} - 0.5)^2 + 10\, x_{i4} + 5\, x_{i5} + x_i^\top \beta,$$

where $\beta_j \overset{\text{i.i.d.}}{\sim} (1-s)\,\delta_0 + s\,\mathcal{N}(0,1)$ with sparsity $s = 0.05$. Within each replicate we draw a fresh β, rescale f to unit empirical variance via a reference draw, and set $\log T_i = f(x_i) + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$. The noise scale σ is chosen so that $\mathrm{var}(\log T_i)/\sigma^2 = 10/9$.

Each training subject is observed at three inspection times $V_{i1} < V_{i2} < V_{i3}$ drawn from $\mathrm{Uniform}(L, U)$, with L a finite floor below the smallest realised $\log T_i$ and U the empirical 95th percentile of $\log T$. The recorded interval is $(V_{i,j-1}, V_{ij}]$ for the smallest j with $V_{ij} \geq \log T_i$, or (V_{i3}, ∞) if $\log T_i$ exceeds the last inspection. The test set of size $n_{\text{test}} = 1{,}000$ is uncensored and supplies the ground-truth f used to evaluate predictions.

We fit three models per replicate on the same data: `SurvivalBART()`, `SurvivalDART()`, and `HorseTrees()` (the latter with `outcome_type = "interval-censored"` and `k = 0.1`). All three use $m = 200$ trees,

`timescale = "log"`, and 5,000 burn-in and 5,000 posterior draws. We sweep $p \in \{500, 5{,}000\}$ at fixed sample size $n = 200$ and run 100 replicates per setting. For each fit we report three metrics on both the training and test samples — comparing the posterior mean and pointwise 95% credible intervals of the regression function against the true f: root mean squared error (RMSE), empirical coverage, and average interval length. The results are given in Table 1. The Horseshoe Forest attains nominal 95% coverage with the lowest RMSE and the tightest credible intervals at both $p = 500$ and $p = 5{,}000$, whereas the standard BART prior over-covers with intervals roughly 60–90% wider and the Dirichlet (DART) prior under-covers. Performance is essentially flat in p, indicating that horseshoe shrinkage scales gracefully into the $p \gg n$ regime.

p	Method	Split	RMSE	Coverage	Length
500	BART	train	1.34 (0.21)	0.971 (0.039)	6.38 (0.45)
		test	1.10 (0.10)	1.000 (0.001)	7.60 (0.71)
	DART	train	1.36 (0.18)	0.894 (0.048)	4.48 (0.41)
		test	1.41 (0.18)	0.898 (0.042)	4.70 (0.47)
	Horseshoe Forest	train	1.00 (0.06)	0.952 (0.027)	3.94 (0.36)
		test	1.03 (0.06)	0.951 (0.029)	4.08 (0.38)
5,000	BART	train	1.35 (0.25)	0.971 (0.046)	6.45 (0.45)
		test	1.08 (0.10)	0.999 (0.002)	7.73 (0.81)
	DART	train	1.38 (0.20)	0.898 (0.044)	4.56 (0.44)
		test	1.46 (0.19)	0.896 (0.039)	4.80 (0.53)
	Horseshoe Forest	train	0.99 (0.07)	0.949 (0.025)	3.94 (0.37)
		test	1.03 (0.06)	0.947 (0.028)	4.07 (0.39)

Table 1: Interval-censored simulation: recovery of the regression function f on training and test samples, averaged over 100 replicates (standard deviation in parentheses). nominal coverage rate is 0.95.

5 Package architecture

ShrinkageTrees is available on CRAN and on GitHub at `https://github.com/tijn-jacobs/ShrinkageTrees`. The R layer wraps a C++ backend through **Rcpp** (Eddelbuettel and François, 2011). We organise the description as follows: (i) fitting functions and S3 classes, (ii) the Rcpp backend, and (iii) workflow features such as multi-chain MCMC, **coda** integration, and plotting.

Fitting functions and S3 classes

The package provides two main fitting functions — one for prediction, one for causal inference — and six convenience wrappers that fix particular model configurations. Table 2 lists all eight functions. The main functions accept arguments that control the outcome type (`outcome_type`, `timescale`), prior specification (`prior_type`, `local_hp`, `global_hp`, and Dirichlet hyperparameters `a_dirichlet`, `b_dirichlet`, `rho_dirichlet`), tree structure (`number_of_trees`, `power`, `base`, `p_grow`, `p_prune`), and MCMC settings (`N_post`, `N_burn`, `n_chains`), among others. Each wrapper preselects a subset of these to expose a smaller, task-specific interface.

`ShrinkageTrees()` is the main prediction model function. It accepts a `prior_type` argument that selects the leaf prior: `"horseshoe"` (per-tree global scale), `"horseshoe_fw"` (forest-wide global scale), `"half-cauchy"` (local shrinkage only), `"standard"` (conjugate normal, as in classical BART), or `"dirichlet"` (Dirichlet splitting, as in DART). `CausalShrinkageForest()` is the main causal-inference function. It mirrors this design for the τ-learner with independent prior configurations for the prognostic and treatment effect forests. The six wrappers preselect a particular `prior_type` and/or `outcome_type` to give a smaller, task-specific interface: `HorseTrees()` and `CausalHorseForest()` fix the prior to the horseshoe and expose a single tuning parameter k controlling the half-Cauchy scales via $\alpha_\tau = \alpha_\lambda = k/\sqrt{m}$. `SurvivalBART()` and `SurvivalDART()` fix the AFT survival outcome with the conjugate-normal and Dirichlet priors, respectively. `SurvivalBCF()` and `SurvivalShrinkageBCF()` do the same for the τ-learner causal model.

All prediction model functions return an object of class `"ShrinkageTrees"` and all causal model functions return `"CausalShrinkageForest"`. Both classes provide `print()`, `summary()`, `predict()`, and `plot()` methods. The `summary()` method reports posterior summaries for σ, variable inclusion

Function	Description
`ShrinkageTrees()`	Prediction model with user-specified leaf prior (horseshoe, half-Cauchy, standard, Dirichlet)
`CausalShrinkageForest()`	τ-learner causal model with user-specified leaf priors for prognostic and treatment forests
`HorseTrees()`	Prediction model with horseshoe prior, with a single tuning parameter k
`CausalHorseForest()`	τ-learner causal model with horseshoe prior on both forests
`SurvivalBART()`	AFT survival with conjugate normal leaf prior (standard BART)
`SurvivalDART()`	AFT survival with Dirichlet splitting prior (DART)
`SurvivalBCF()`	AFT survival with τ-learner decomposition and conjugate normal leaf prior
`SurvivalShrinkageBCF()`	AFT survival with τ-learner decomposition and horseshoe leaf prior

Table 2: Fitting functions in **ShrinkageTrees**. The first two are the main functions. The remaining six are convenience wrappers that fix particular model configurations.

probabilities, and — for causal models — the ATE with a credible interval. The `predict()` method re-runs the MCMC sampler on the new covariate matrix using the training data and hyperparameters stored in the fitted object. It always generates posterior samples internally, regardless of whether `store_posterior_sample` was set during fitting. The returned `ShrinkageTreesPrediction` object contains posterior means and credible intervals. The `plot()` method produces MCMC diagnostics (traceplots and posterior densities for σ), variable importance plots, and — for causal models — ATE and CATE visualisations. Plotting uses **ggplot2**, a suggested but not required dependency.

Rcpp backend

We implement the computationally intensive components of the Gibbs sampler in C++ via **Rcpp**. The C++ layer covers the Metropolis–Hastings updates for the tree topology (grow, prune, and change moves), the conditional updates for the leaf step heights, the augmentation steps for the local and global scale parameters under horseshoe shrinkage, the update for σ, and the data augmentation for censored event times. The R layer handles input validation, data preprocessing (centring, scaling, log-transformation of survival times), hyperparameter calibration, S3 class construction, and post-processing (back-transformation of predictions).

The C++ backend dispatches between two forest implementations depending on the leaf prior. The `Forest` class implements reversible jump moves that update the tree and its step heights jointly. It is used for the `"horseshoe"`, `"horseshoe_fw"`, and `"half-cauchy"` priors. The `StanForest` class implements moves that update the tree structure and the step heights sequentially. It is used for the `"standard"` and `"dirichlet"` priors. The latter approach is simpler and faster, but applies only when the leaf prior is conjugate and global scale updates are unnecessary.

A single C++ class, `ScaleMixture`, holds the global–local step height prior and is queried by the tree updates whenever a new step height is proposed. The current implementations are `FixedVariance` (conjugate normal, used by BART and DART), `HalfCauchy` (local shrinkage only), `Horseshoe` (per-tree global–local shrinkage), and `Horseshoe_fw` (forest-wide global shrinkage). The user's `prior_type` argument determines which implementation is constructed at run time. Because the tree update and reversible jump routines interact with the prior only through the `ScaleMixture` interface, a developer can add a new global–local shrinkage prior — for example a regularised horseshoe or a Dirichlet–Laplace prior — by adding a new implementation and registering it. The sampler logic does not need to change.

Figure 8 summarises the package architecture.

Multi-chain MCMC

All fitting functions accept an `n_chains` argument. When `n_chains > 1`, the package dispatches independent chains in parallel via `parallel::mclapply()`. On Windows, `mclapply()` falls back to sequential execution. The package pools the draws from all chains and recomputes posterior sum-

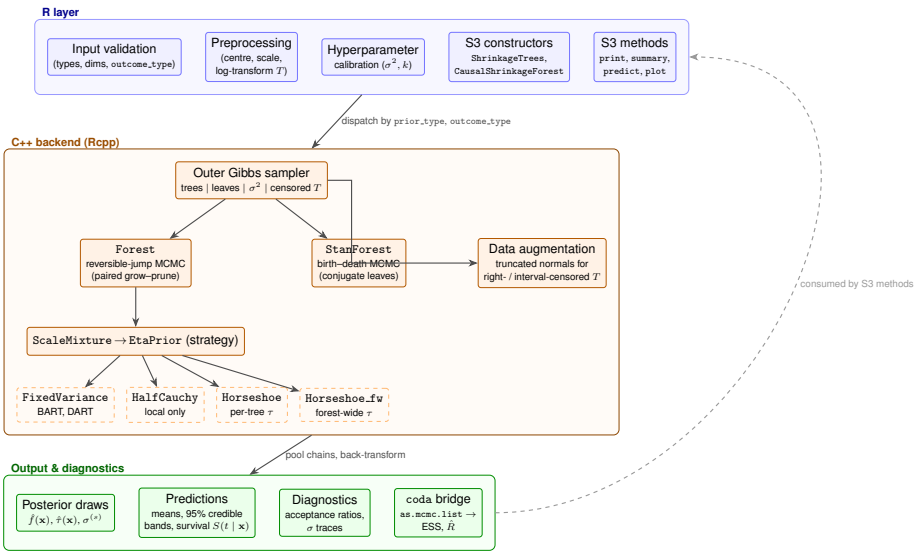

Figure 8: Package architecture. The R layer (top) handles input validation, preprocessing, hyperparameter calibration, S3 construction, and post-processing methods. The C++ backend (middle) runs the outer Gibbs sampler, dispatching to the reversible-jump `Forest` class or the conjugate birth–death `StanForest` class depending on the leaf prior. The `ScaleMixture` wrapper selects one of four `EtaPrior` subclasses at runtime via a factory. The output layer (bottom) exposes posterior draws, predictions, diagnostics, and a **coda** bridge to the user. **Note to self: revise or delete graph.**

maries; the returned object is a standard `"ShrinkageTrees"` or `"CausalShrinkageForest"` instance, so all downstream methods work without modification. Per-chain acceptance ratios and σ traces remain available for convergence assessment.

Convergence diagnostics via coda

We assess MCMC convergence via the **coda** package (Plummer et al., 2006). The package exposes posterior draws to **coda** through an `as.mcmc.list()` S3 method. The method extracts the posterior draws of σ, splits them by chain, wraps each chain in a `coda::mcmc` object, and returns a `coda::mcmc.list`. **coda** is a suggested rather than required dependency; the package's other functionality does not depend on it. When **coda** is installed, `summary()` reports the effective sample size (ESS) and, for multi-chain fits, the Gelman–Rubin $\hat{R}$ statistic. Users who require more detailed diagnostics can pass the `mcmc.list` object directly to any **coda** function, including `geweke.diag()`, `heidel.diag()`, `autocorr.plot()`, and `gelman.plot()`.

Computational speed

We benchmark the wall-clock cost of fitting survival models in **ShrinkageTrees** and compare it with **BART**, the only other CRAN package that provides Bayesian tree ensembles for survival data. All timings are measured on a 2024 MacBook Air with an Apple M3 chip and 16 GB of memory, running R version 4.5.2, **ShrinkageTrees** 2.0.2, and **BART** 2.9.10. Each fit uses `n_chains = 4`, $m = 200$ trees, and 1000 posterior draws per chain.

We compare three fitting functions in **ShrinkageTrees** — `SurvivalBART()`, `SurvivalDART()`, and `HorseTrees()` — against `abart()` from **BART** on simulated right-censored survival data. `abart()` does not have an `n_chains` argument. We therefore use its multi-chain wrapper `mc.abart()` with `mc.cores = 4`, which runs four independent chains in parallel via `parallel::mcparallel`, mirroring how **ShrinkageTrees** dispatches its own chains.

Figure 9 shows how wall-clock time scales with the sample size n, with $p = 100$ and $m = 200$ fixed. Cost scales approximately linearly in n for all four fitting functions. The conjugate-leaf models (`SurvivalBART()` and `SurvivalDART()`) are the closest comparators to `abart()` and run at similar cost. The horseshoe variant (`HorseTrees()`) is more expensive because it combines reversible jump tree updates with per-leaf local scale draws.

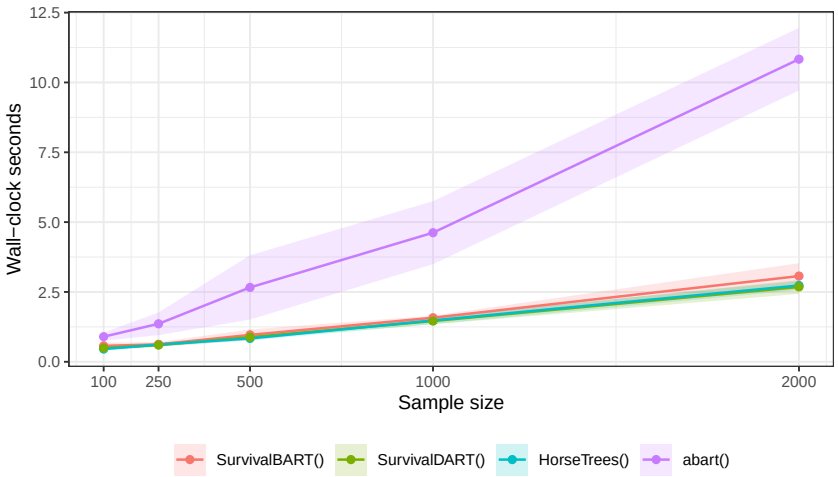

Figure 9: Wall-clock seconds vs sample size n for the four fitting functions, with $p = 100$ and $m = 200$ held fixed and `n_chains` $= 4$ throughout. Lines and points show means over three replications. Shaded ribbons show ± 1 SD.

6 Related packages

Several R packages implement Bayesian tree ensembles. We review five comparator packages and contrast their capabilities with **ShrinkageTrees**. Table 3 provides a summary.[1]

BART (Sparapani et al., 2021) is the most general-purpose package in this comparison and covers the widest range of outcome types. It supports continuous, binary, and multinomial outcomes, and is the only other CRAN package to offer Bayesian tree ensembles for survival data. Survival models include a discrete-time hazard formulation (`surv.bart()`), an AFT formulation (`abart()`), competing risks (`crisk.bart()`), and recurrent events (`recur.bart()`). The package also supports Dirichlet splitting priors via `sparse = TRUE`. **BART** does not support global–local shrinkage priors on the leaf step heights, causal forest models, or interval-censored survival outcomes. Multi-chain MCMC requires the prefixed `mc.*` wrappers (e.g. `mc.abart()`).

dbarts (Dorie, 2023) provides a BART sampler for continuous and binary outcomes. Its key feature is an updatable sampler object: predictors and responses can be modified mid-sampling, which makes **dbarts** well suited as a conditional model within a larger Gibbs sampler. The package supports multi-chain MCMC but does not support survival models, τ-learner models, or global–local shrinkage on the leaf step heights.

bcf (Hahn et al., 2020) implements the τ-learner with separate prognostic and treatment-effect forests for continuous outcomes. It places half-Cauchy and half-Normal hyperpriors on the leaf scales of the two forests, respectively. The package does not support survival endpoints, global–local shrinkage priors, or Dirichlet splitting.

stochtree (Herren et al., 2024) implements BART, XBART (He et al., 2019), the τ-learner, and XBCF for continuous, binary, and ordinal outcomes. It supports variance forests for heteroskedastic models and serialises fitted objects so they can be reused as warm starts. **stochtree** does not support survival models, Dirichlet splitting priors, or global–local shrinkage on the leaf step heights.

SoftBart (Linero and Yang, 2018) replaces the hard axis-aligned splits of standard BART with smooth logistic splitting rules, so the posterior mean is a continuous function of the covariates. Regularisation is achieved through a Dirichlet prior on the variable inclusion probabilities (the same mechanism as DART) rather than through priors on the leaf step heights. This combination can improve performance when the true regression surface is smooth, but introduces additional computational cost. **ShrinkageTrees** retains hard decision splits, as in classical BART, and instead places shrinkage priors on the leaf step heights themselves. **SoftBart** does not support survival endpoints, the τ-learner, or global–local shrinkage on the leaf step heights.

Across these packages, **ShrinkageTrees** occupies a distinct niche: survival outcomes (right- and interval-censored), the τ-learner for survival, and global–local shrinkage on the leaf step heights.

[1]Package descriptions reflect CRAN versions: BART 2.9.10, dbarts 0.9-33, bcf 2.0.2, stochtree 0.4.2, SoftBart 1.0.3.

Package	Survival	Interval censoring	Causal (τ-learner)	Dirichlet (DART)	Global–local shrinkage
BART	Yes	No	No	Yes	No
dbarts	No	No	No	No	No
bcf	No	No	Yes	No	No
stochtree	No	No	Yes	No	No
SoftBart	No	No	No	No	No
ShrinkageTrees	Yes	Yes	Yes	Yes	Yes

Table 3: Comparison of Bayesian tree ensemble packages in R.

7 Discussion and conclusion

ShrinkageTrees brings global–local shrinkage priors to Bayesian tree ensembles for survival analysis and causal inference. The package separates regularisation of the tree structure from regularisation of the leaf step heights, so practitioners can tailor the regularisation strategy to the problem at hand. It is the first CRAN package to offer the τ-learner for survival endpoints, horseshoe priors on tree ensemble step heights, and interval-censored survival outcomes within a single framework.

The package has several limitations that suggest directions for future work. All covariates must be numeric. The axis-aligned splitting rules handle continuous, binary, and ordered categorical variables (encoded as integers) directly, but the user must dummy-encode unordered categorical variables. This can be suboptimal for high-cardinality factors, where native subset splits would be more efficient. The current AFT formulation assumes normally distributed errors. Alternative parametric error distributions, such as the logistic or log-logistic, would broaden the package's applicability. Non-parametric error models, for example those built on stick-breaking process priors, are a further direction.

ShrinkageTrees is available on CRAN and on GitHub at `https://github.com/tijn-jacobs/ShrinkageTrees`, where bug reports and feature requests can be filed via the issue tracker at `https://github.com/tijn-jacobs/ShrinkageTrees/issues`. Jacobs et al. (2025) develops the underlying methodology.

8 Funding

Tijn Jacobs and Stéphanie van der Pas received funding from the European Research Council (ERC) under the European Union's Horizon Europe program under Grant agreement No. 101074082. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.

Bibliography

Cancer Genome Atlas Research Network. Integrated genomic analyses of ovarian carcinoma. *Nature*, 474:609–615, 2011. doi: 10.1038/nature10166. [p1, 2]

A. Caron, G. Baio, and I. Manolopoulou. Estimating individual treatment effects using non-parametric regression models: A review. *Journal of the Royal Statistical Society Series A: Statistics in Society*, 185(3): 1115–1149, 2022. doi: 10.1111/rssa.12889. [p1, 6]

C. M. Carvalho, N. G. Polson, and J. G. Scott. The horseshoe estimator for sparse signals. *Biometrika*, 97(2):465–480, 2010. doi: 10.1093/biomet/asq017. [p1, 5]

H. A. Chipman, E. I. George, and R. E. McCulloch. Bayesian CART model search. *Journal of the American Statistical Association*, 93(443):935–948, 1998. doi: 10.1080/01621459.1998.10473750. [p4]

H. A. Chipman, E. I. George, and R. E. McCulloch. BART: Bayesian additive regression trees. *The Annals of Applied Statistics*, 4(1):266–298, 2010. doi: 10.1214/09-AOAS285. [p1, 2, 4]

V. Dorie. *dbarts: Discrete Bayesian Additive Regression Trees Sampler*, 2023. URL `https://CRAN.R-project.org/package=dbarts`. R package version 0.9-23. [p15]

V. Dorie, J. Hill, U. Shalit, M. Scott, and D. Cervone. Automated versus do-it-yourself methods for causal inference: Lessons learned from a data analysis competition. *Statistical Science*, 34(1):43–68, 2019. doi: 10.1214/18-STS667. [p6]

D. Eddelbuettel and R. François. Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, 40(8):1–18, 2011. doi: 10.18637/jss.v040.i08. [p1, 12]

J. H. Friedman. Multivariate adaptive regression splines. *The Annals of Statistics*, 19(1):1–67, 1991. doi: 10.1214/aos/1176347963. [p11]

P. R. Hahn, J. S. Murray, and C. M. Carvalho. Bayesian regression tree models for causal inference: Regularization, confounding, and heterogeneous effects. *Bayesian Analysis*, 15(3):965–1056, 2020. doi: 10.1214/19-BA1195. [p1, 6, 15]

F. E. Harrell, K. L. Lee, and D. B. Mark. Multivariable prognostic models: Issues in developing models, evaluating assumptions and adequacy, and measuring and reducing errors. *Statistics in Medicine*, 15 (4):361–387, 1996. doi: 10.1002/(SICI)1097-0258(19960229)15:4<361::AID-SIM168>3.0.CO;2-4. [p2]

T. Hastie and R. Tibshirani. Bayesian backfitting (with comments and a rejoinder by the authors). *Statistical Science*, 15(3):196–223, 2000. doi: 10.1214/ss/1009212815. [p6]

J. He, S. Yalov, and P. R. Hahn. XBART: Accelerated Bayesian additive regression trees. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 89, pages 1130–1138, 2019. URL https://proceedings.mlr.press/v89/he19a.html. [p15]

D. Herren, J. He, and R. Hahn. *stochtree: Stochastic Tree Ensembles (XBART and BART) for Supervised Learning and Causal Inference*, 2024. URL https://CRAN.R-project.org/package=stochtree. R package. [p15]

J. L. Hill. Bayesian nonparametric modeling for causal inference. *Journal of Computational and Graphical Statistics*, 20(1):217–240, 2011. doi: 10.1198/jcgs.2010.08162. [p6]

T. Jacobs, W. N. van Wieringen, and S. L. van der Pas. Horseshoe forests for high-dimensional causal survival analysis. *arXiv preprint arXiv:2507.22004*, 2025. doi: 10.48550/arXiv.2507.22004. [p1, 5, 7, 16]

A. Kapelner and J. Bleich. bartMachine: Machine learning with Bayesian additive regression trees. *Journal of Statistical Software*, 70(4):1–40, 2016. doi: 10.18637/jss.v070.i04. [p6]

F. Li, P. Ding, and F. Mealli. Bayesian causal inference: A critical review. *Philosophical Transactions of the Royal Society A*, 381(2247):20220153, 2023. doi: 10.1098/rsta.2022.0153. [p11]

A. R. Linero. Bayesian regression trees for high-dimensional prediction and variable selection. *Journal of the American Statistical Association*, 113(522):626–636, 2018. doi: 10.1080/01621459.2016.1264957. [p1, 5]

A. R. Linero and Y. Yang. Bayesian regression tree ensembles that adapt to smoothness and sparsity. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 80(5):1087–1110, 2018. doi: 10.1111/rssb.12293. [p15]

M. Plummer, N. Best, K. Cowles, and K. Vines. CODA: Convergence diagnosis and output analysis for MCMC. *R News*, 6(1):7–11, 2006. URL https://journal.r-project.org/archive/. [p14]

N. G. Polson and J. G. Scott. On the half-Cauchy prior for a global scale parameter. *Bayesian Analysis*, 7 (4):887–902, 2012. doi: 10.1214/12-BA730. [p5]

V. Ročková and E. Saha. On theory for BART. In K. Chaudhuri and M. Sugiyama, editors, *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 2839–2848. PMLR, 16–18 Apr 2019. URL https://proceedings.mlr.press/v89/rockova19a.html. [p4]

D. B. Rubin. The Bayesian bootstrap. *The Annals of Statistics*, 9(1):130–134, 1981. doi: 10.1214/aos/1176345338. [p10]

R. Sparapani, C. Spanbauer, and R. McCulloch. Nonparametric machine learning and efficient computation with Bayesian additive regression trees: The BART R package. *Journal of Statistical Software*, 97(1):1–66, 2021. doi: 10.18637/jss.v097.i01. [p15]

Tijn Jacobs
Vrije Universiteit Amsterdam
Department of Mathematics
Amsterdam, The Netherlands
https://tijn-jacobs.github.io/
ORCiD: 0009-0003-6188-9296
t.jacobs@vu.nl

Stéphanie van der Pas
Vrije Universiteit Amsterdam
Department of Mathematics
Amsterdam, The Netherlands
https://www.stephanievanderpas.nl/
ORCiD: 0000-0002-2448-5378
s.l.vander.pas@vu.nl

Wessel van Wieringen
Vrije Universiteit Amsterdam\ Amsterdam University Medical Centre
Department of Mathematics, Amsterdam, The Netherlands
Epidemiology and Data Science, Amsterdam, The Netherlands
https://www.math.vu.nl/~wvanwie/
ORCiD: 0000-0002-5100-9123
w.n.vanwieringen@vu.nl